

HỒI QUY LOGISTIC

Logistic Regression — Phân loại nhị phân & đa lớp

TS. Cao Tiến Dũng



**FPT SCHOOL OF BUSINESS
& TECHNOLOGY**

Vì sao cần Logistic Regression?

- Nhiều bài toán thực tế cần **PHÂN LOẠI** dữ liệu vào các hạng mục, không phải dự đoán một số liên tục.
- Ví dụ phân loại trong thực tế:
 - **Lọc thư rác** — email này có phải thư rác?
 - **Chẩn đoán bệnh** — bệnh nhân có mắc bệnh hay không?
 - **Phát hiện gian lận** — giao dịch này có gian lận?
 - **Rời bỏ khách hàng** — khách hàng sẽ rời đi hay ở lại?

⚠ **Dù tên là “hồi quy”,** Logistic Regression được dùng để **PHÂN LOẠI** — là thuật toán nền tảng cho phân loại nhị phân và đa lớp.

Từ hồi quy tuyến tính → Logistic

- Hồi quy = mô hình hóa mức độ liên hệ giữa biến đầu vào và đầu ra. Đầu ra có thể LIÊN TỤC hoặc RỜI RẠC (hạng mục).

Hồi quy tuyến tính

- Đầu ra LIÊN TỤC (một số).
- *Ví dụ: giá nhà, nhiệt độ.*

Hồi quy Logistic

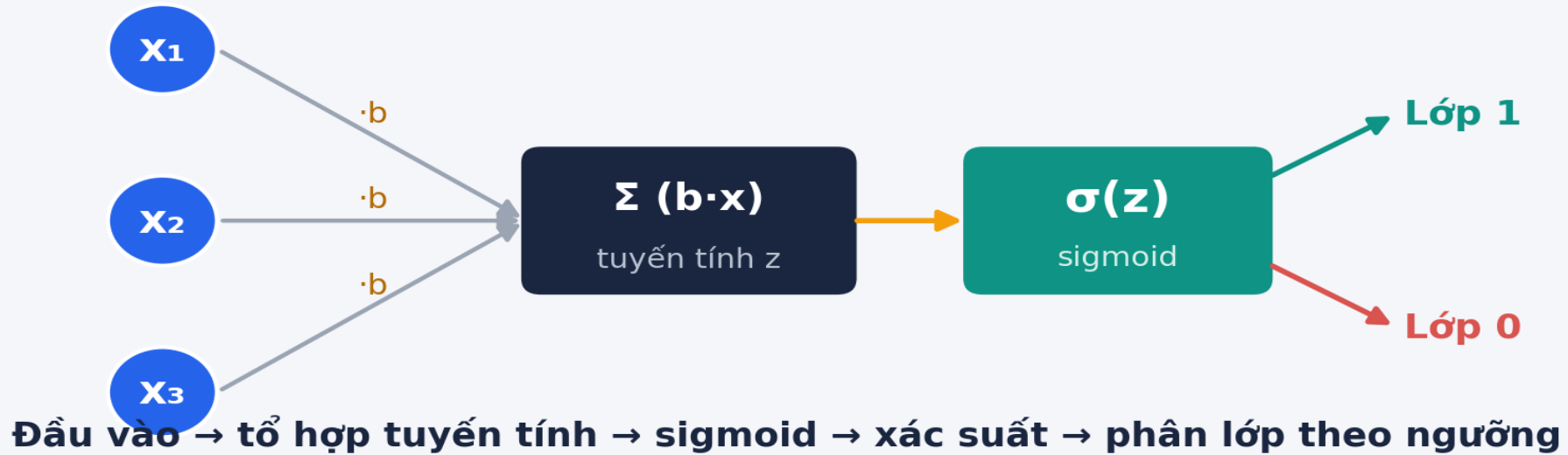
- Đầu ra RỜI RẠC → phân lớp.
- **Nhị phân:** 2 lớp.
- **Đa thức (multinomial):** hơn 2 lớp.

01

Mô hình Logistic nhị phân

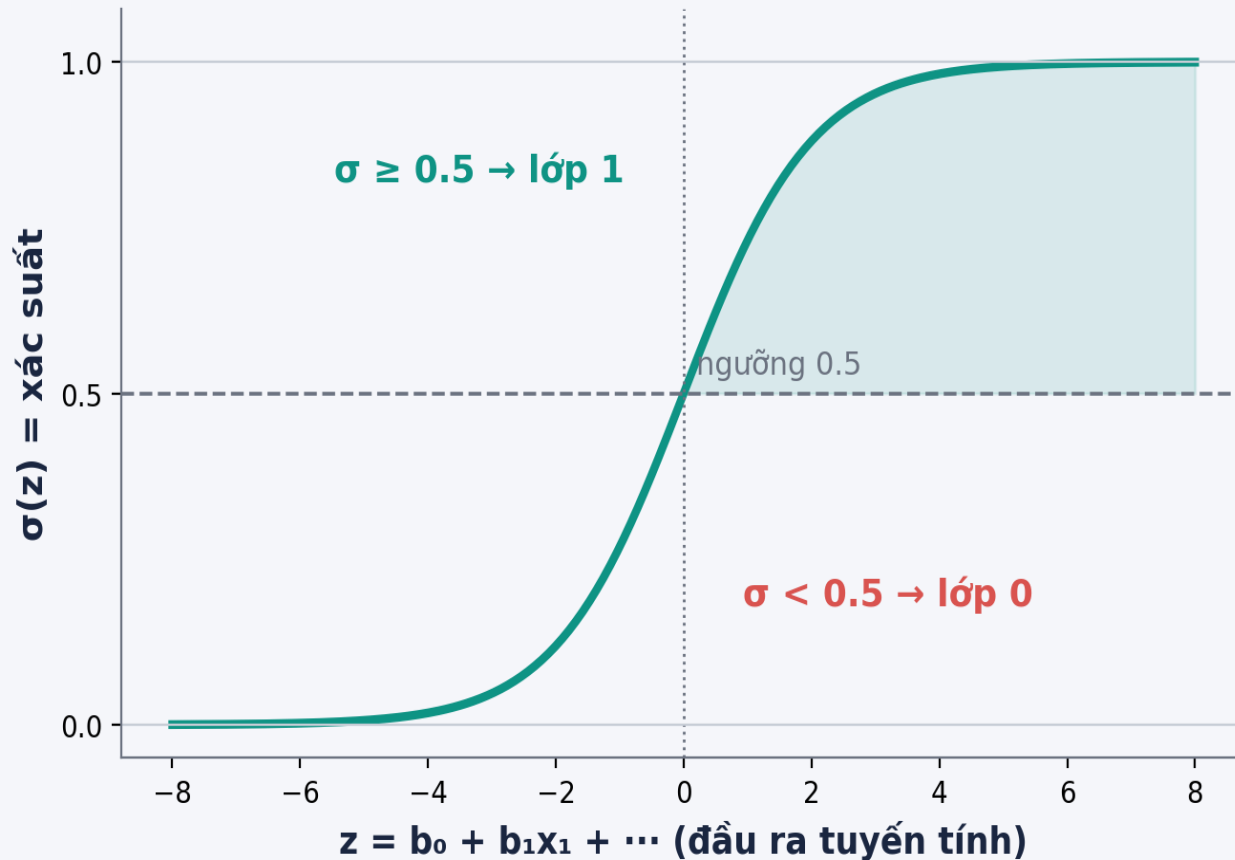
Binary Logistic Regression

Mô hình Logistic



- **Bước 1 — Tuyến tính:** tính $z = b_0 + b_1x_1 + \dots + b_px_p$ (như hồi quy tuyến tính).
- **Bước 2 — Sigmoid:** ép z về xác suất trong $(0,1)$; so với ngưỡng để ra lớp 0 / lớp 1.

Hàm Sigmoid



$$\sigma(z) = \frac{1}{1 + e^{-z}}, \quad z = b_0 + b_1x_1 + \dots + b_px_p$$

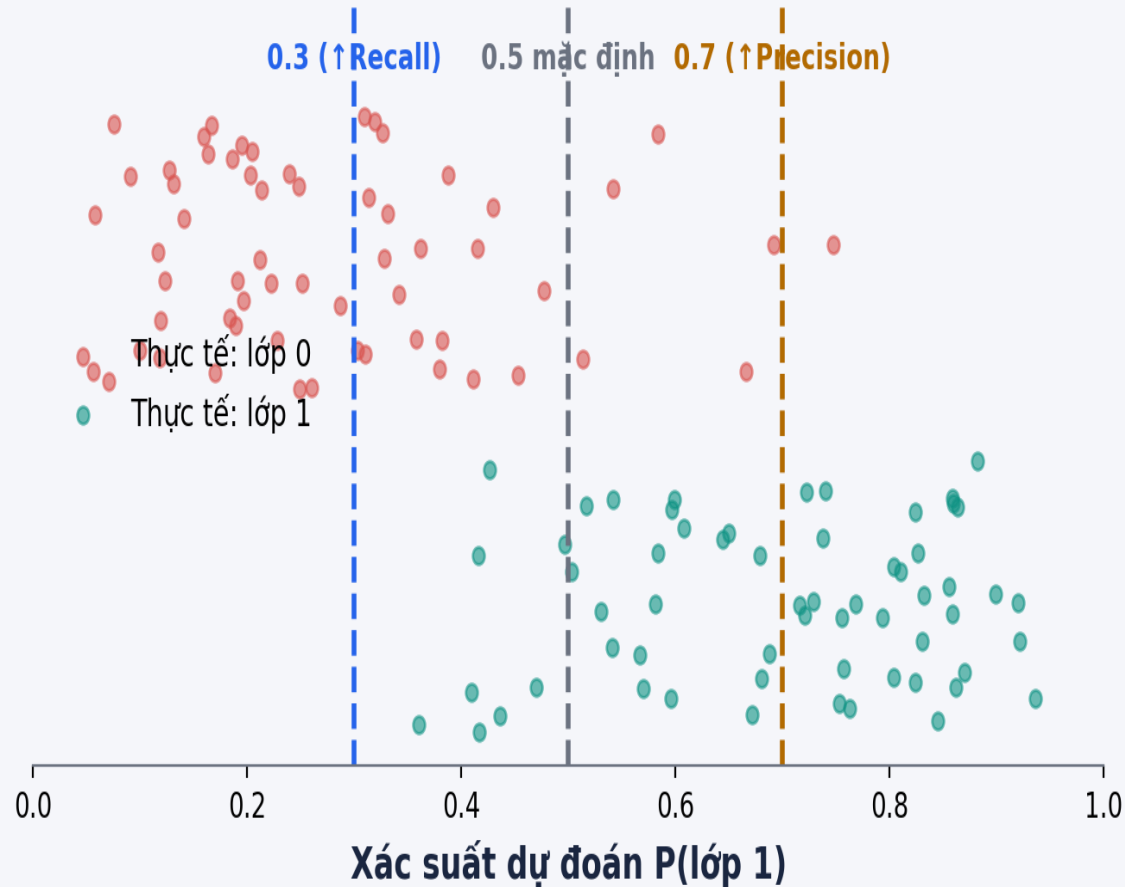
- Biến đổi đầu ra tuyến tính z (bất kỳ trong $\mathbb{R}$) thành XÁC SUẤT trong $(0, 1)$.

- **Dạng logit (nghịch đảo):**

$$\log\left(\frac{p}{1-p}\right) = b_0 + b_1x_1 + \dots + b_px_p$$

log-odds là tuyến tính theo $x \rightarrow$ diễn giải được hệ số.

Ngưỡng quyết định (Decision Threshold)



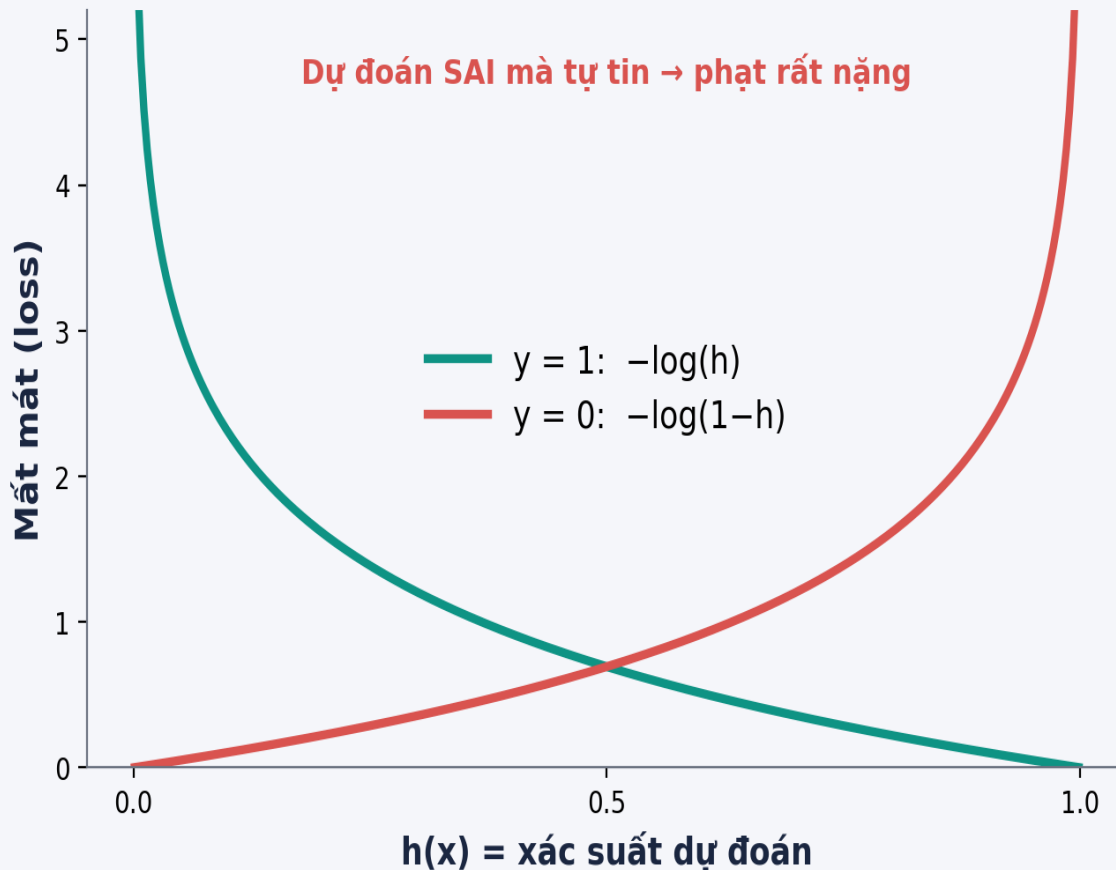
- **Quy tắc:** nếu $P(\text{lớp 1}) \geq \text{ngưỡng} \rightarrow \text{lớp 1}$, ngược lại lớp 0. Mặc định ngưỡng = 0.5.
- **Hạ ngưỡng (vd 0.3):** bắt nhiều ca dương hơn $\rightarrow$ tăng Recall, giảm Precision.
- **Nâng ngưỡng (0.7):** thận trọng hơn $\rightarrow$ tăng Precision, giảm Recall.
- *Chọn ngưỡng theo chi phí sai lầm của bài toán, không nhất thiết 0.5.*

02

Hàm mất mát & Tối ưu hóa

Loss Function & Optimization

Vì sao Log Loss, không phải MSE?



- **Nếu dùng MSE với sigmoid:** mặt chi phí KHÔNG lồi (nhiều cực tiểu địa phương), gradient descent dễ kẹt, hội tụ chậm gần 0 và 1.
- **Binary Cross-Entropy (Log Loss):** mặt chi phí LỖI → một cực tiểu toàn cục duy nhất; luôn hội tụ; phạt nặng dự đoán sai mà tự tin.

$$L = -y \log(h(x)) - (1 - y) \log(1 - h(x))$$

Hàm chi phí — Binary Cross-Entropy

- Gọi $h(x) = g(\mathbf{b}^\top x)$ là xác suất x thuộc lớp 1. Chi phí cho một mẫu:

$$Cost(g(\mathbf{b}^\top x), y) = \begin{cases} -\log(g(\mathbf{b}^\top x)) & y = 1 \\ -\log(1 - g(\mathbf{b}^\top x)) & y = 0 \end{cases}$$

- **Khi $y=1$:** phạt khi $h \rightarrow 0$ (dự đoán sai về lớp 1).
- **Khi $y=0$:** phạt khi $h \rightarrow 1$ (dự đoán sai về lớp 0).
- Gộp lại thành công thức Log Loss đã nêu; lấy trung bình trên toàn bộ mẫu $\rightarrow$ hàm chi phí $J(b)$ cần tối thiểu.

Gradient Descent & Cập nhật trọng số

- Áp dụng quy tắc chuỗi (chain rule) cho đạo hàm của Log Loss qua sigmoid, kết quả rút gọn rất gọn đẹp:

Gradient

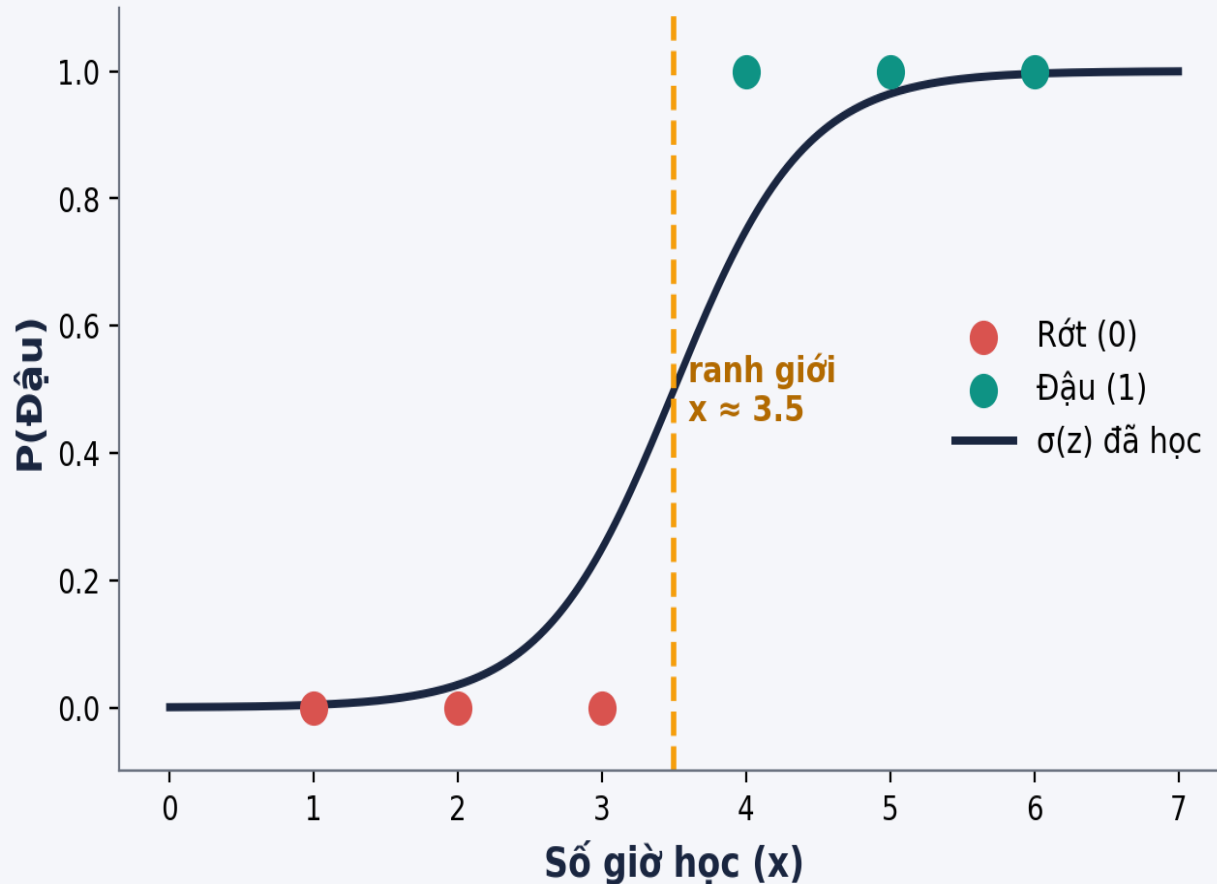
$$\frac{\partial L}{\partial b_j} = (\sigma(z) - y) x_j$$

Quy tắc cập nhật

$$b_j := b_j - \alpha (\sigma(z) - y) x_j$$

- $\sigma(z) - y$: sai số (xác suất dự đoán trừ nhãn thật).
- α : tốc độ học. Lặp cho đến khi hội tụ (chi phí lỗi $\rightarrow$ đảm bảo về cực tiểu toàn cục).
- *Lưu ý: dạng cập nhật giống hồi quy tuyến tính, chỉ khác h là sigmoid thay vì tuyến tính.*

Ví dụ nhỏ: Đậu hay Rớt?



- **Dữ liệu:** số giờ học (x) vs kết quả (y : 0=Rớt, 1=Đậu).

- **Diễn giải huấn luyện:**

- B1: khởi tạo $b_0=0$, $b_1=0$.
- B2: với $x=1 \rightarrow z=0$, $\sigma(0)=0.5$.
- B3: sai số = $0.5 - 0 = 0.5$.
- B4: cập nhật $b_1 = 0 - 0.1 \cdot 0.5 \cdot 1 = -0.05$.
- B5: lặp qua mọi mẫu, nhiều epoch.

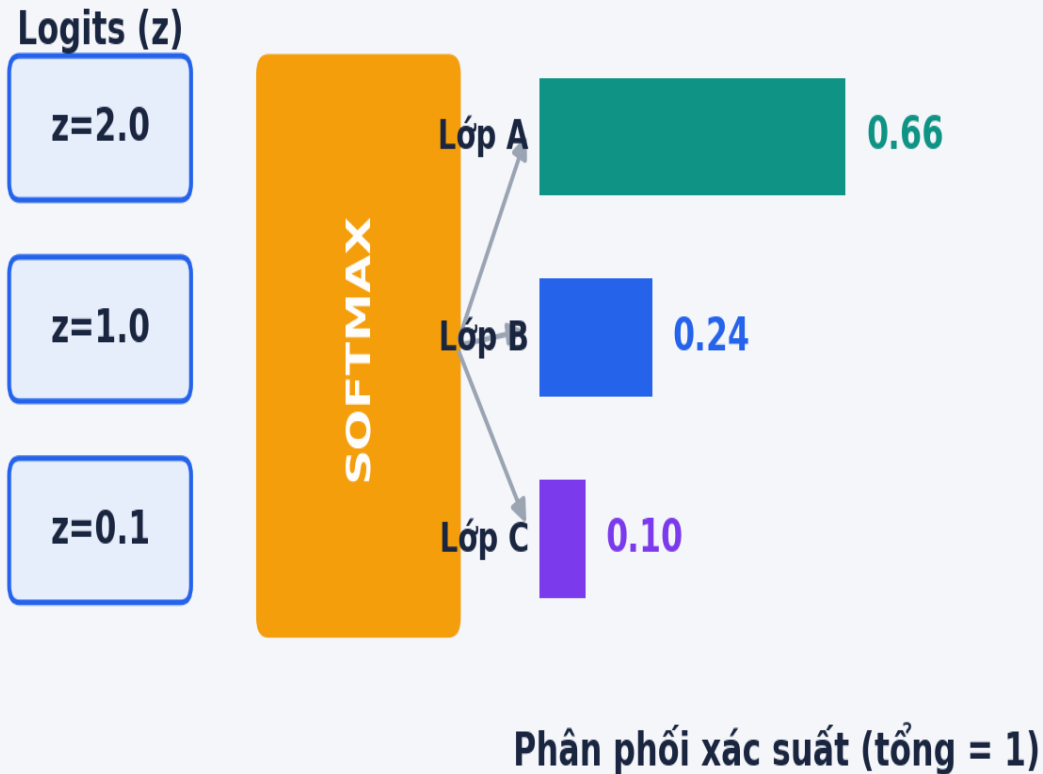
→ Mô hình học được ranh giới quyết định quanh $x \approx 3.5$.

03

Phân loại đa lớp

Multiclass Classification

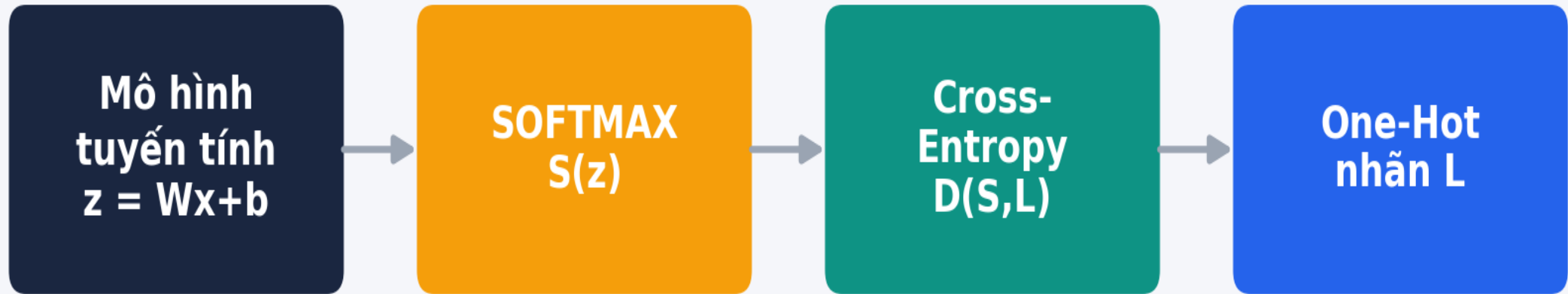
Softmax (Hồi quy Logistic đa thức)



$$\sigma(\mathbf{z})_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}} \quad (i = 1, \dots, K)$$

- Nhận một véc-tơ K số (logits) và chuẩn hóa thành PHÂN PHỐI XÁC SUẤT gồm K xác suất.
- **Tổng = 1**; lớp có xác suất cao nhất được chọn.
- Khi $K=2$, softmax suy biến về sigmoid.

Bộ phân loại Logistic đa lớp



Đầu ra K xác suất → so với nhãn one-hot bằng cross-entropy

- **Luồng xử lý:** mô hình tuyến tính ($z = Wx + b$) → Softmax cho ra K xác suất → so với nhãn one-hot bằng hàm Cross-Entropy → cập nhật trọng số.
- *Mỗi lớp có một bộ trọng số riêng; toàn bộ huấn luyện đồng thời (joint).*

One-vs-Rest vs Softmax



One-vs-Rest (OvR)

- Huấn luyện K bộ phân loại NHỊ PHÂN: “lớp i vs phần còn lại”.
- Chọn lớp có xác suất cao nhất. Đơn giản, dùng được với mọi bộ phân loại nhị phân.

Softmax (Multinomial)

- MỘT mô hình duy nhất, tối ưu chung trên K lớp.
- Xác suất nhất quán (tổng=1), thường chính xác hơn cho bài toán loại trừ lẫn nhau.

04

Đánh giá mô hình phân loại

Classification Metrics

Ma trận nhầm lẫn (Confusion Matrix)

- Nền tảng của mọi độ đo phân loại — đối chiếu nhãn dự đoán với nhãn thực tế.

	Dự đoán: Dương	Dự đoán: Âm
Thực tế: Dương	TP (Dương thật)	FN — Lỗi loại II (bỏ sót dương)
Thực tế: Âm	FP — Lỗi loại I (báo động giả)	TN (Âm thật)

- **TP/TN**: dự đoán đúng.
- **FP (Lỗi loại I)**: báo dương nhầm.
- **FN (Lỗi loại II)**: bỏ sót ca dương.
- Mọi độ đo (*Accuracy, Precision, Recall, F1*) đều suy ra từ 4 ô này.

Accuracy, Precision & Recall

Accuracy:

dùng khi dữ liệu CÂN BẰNG

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

$$Precision = \frac{TP}{TP + FP}$$

$$Recall = \frac{TP}{TP + FN}$$

Precision (Độ chính xác dương)

- Dùng khi muốn RẤT CHẮC về dự đoán dương — “trong số gần cò, bao nhiêu đúng?”

Recall (Độ bao phủ)

- Dùng khi muốn BẮT càng nhiều ca dương càng tốt — “trong ca dương thật, bắt được bao nhiêu?”

F1-Score

$$F_1 = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall}$$

- **Khi nào dùng:** khi muốn mô hình tốt CẢ Precision LÃN Recall.
- *F1 là trung bình điều hòa (harmonic mean) của Precision và Recall — phạt nặng khi hai chỉ số lệch nhau nhiều.*

Khi nào dùng độ đo nào?

Ưu tiên Precision

- Giảm thiểu False Positive.
- Lọc thư rác — tránh chặn nhầm email quan trọng.

Ưu tiên Recall

- Giảm thiểu False Negative.
- Tầm soát ung thư — không bỏ sót ca bệnh nào.

F1 (Cân bằng)

- Hiệu năng cân bằng P–R.
- Phát hiện gian lận — vừa bắt đúng, vừa không chặn nhầm.

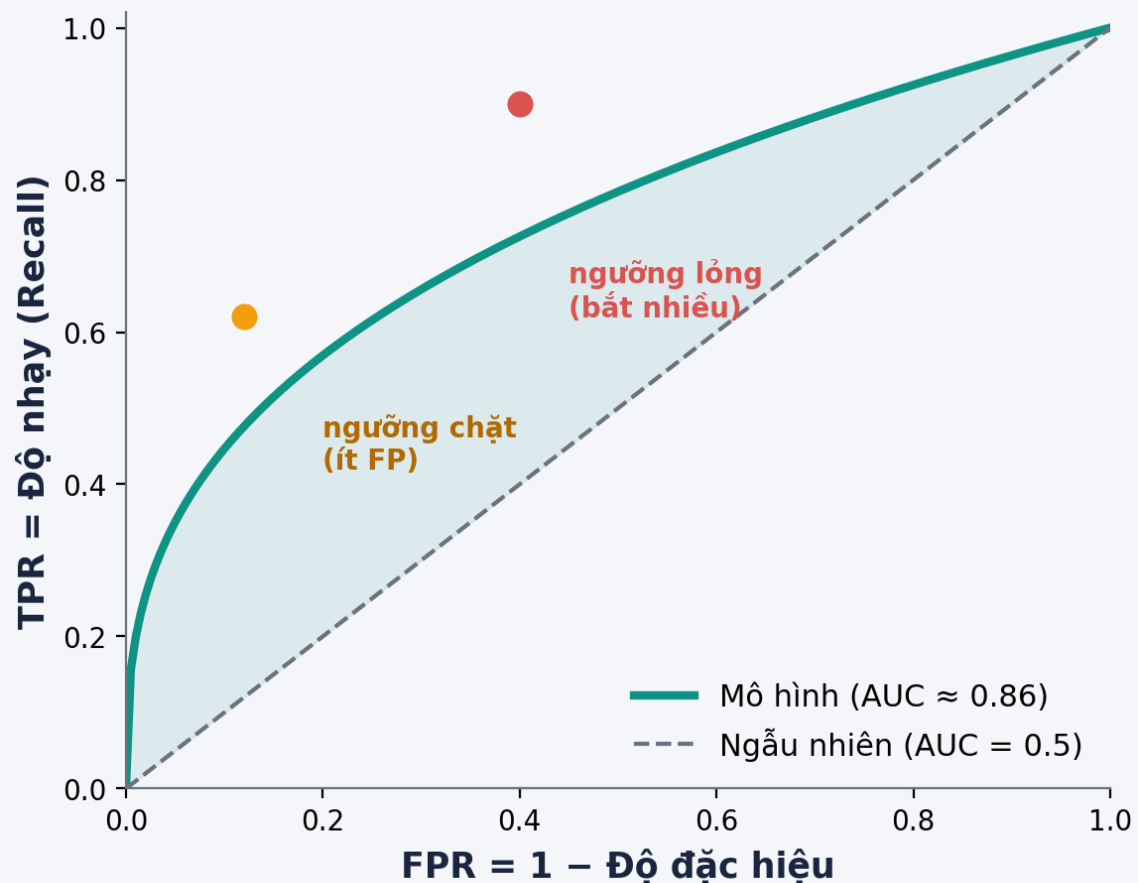
AUC-ROC

- Độc lập ngưỡng, để so sánh mô hình.
- 0.9–1.0 Xuất sắc · 0.8–0.9 Tốt · 0.5 = ngẫu nhiên.

Accuracy

- Chỉ hợp dữ liệu CÂN BẰNG.
- Đánh lừa khi lệch lớp (vd 99% âm, 1% dương).

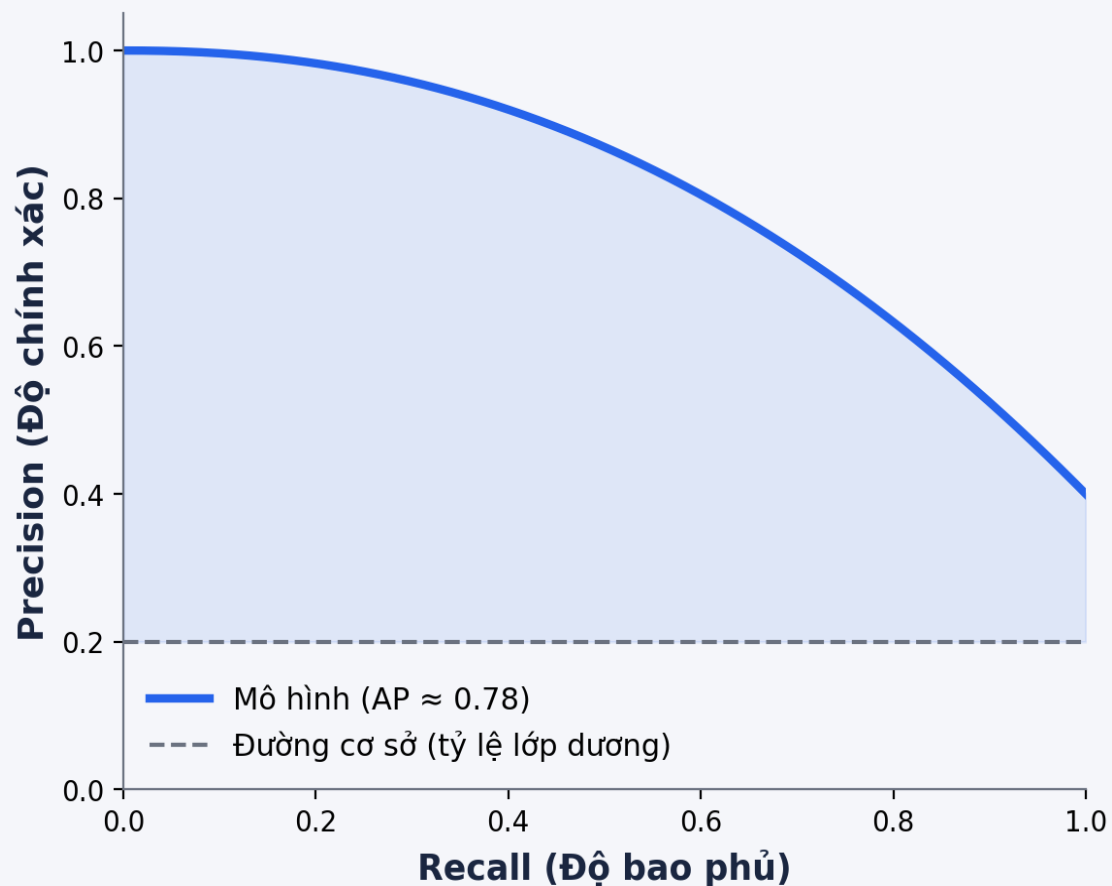
Đường cong ROC & AUC



$$TPR = \frac{TP}{TP + FN} \quad FPR = \frac{FP}{TN + FP}$$

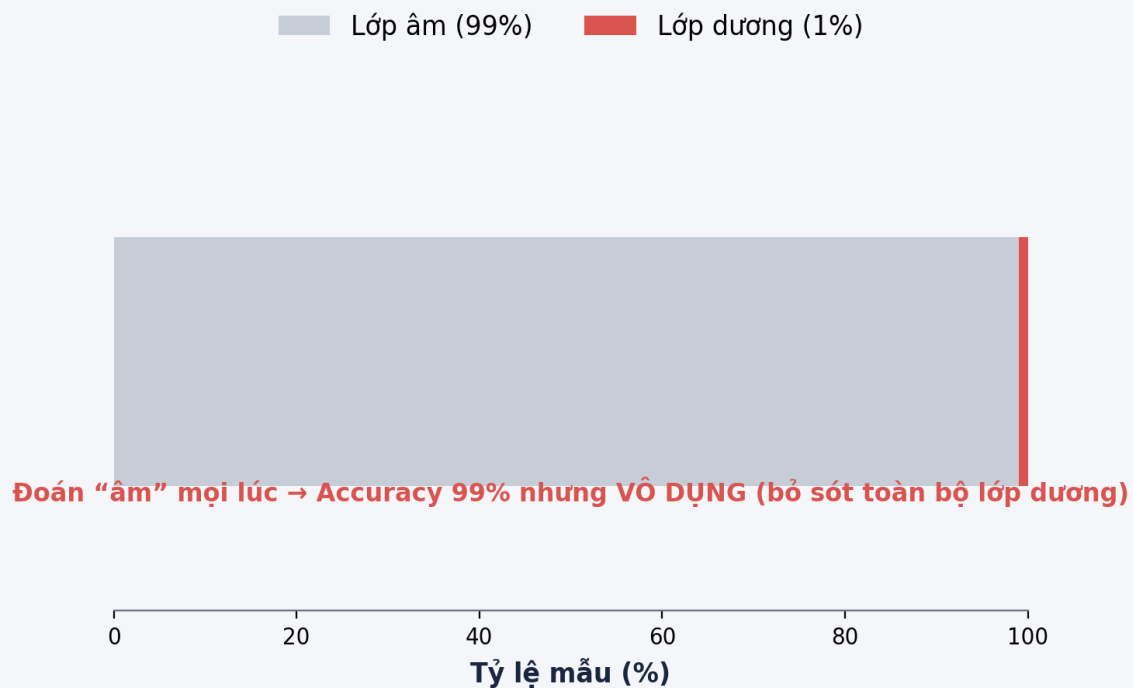
- **ROC:** đồ thị TPR (độ nhạy) theo FPR khi quét mọi ngưỡng.
- **AUC:** diện tích dưới ROC — khả năng phân biệt tổng thể, ĐỘC LẬP ngưỡng.
- *1.0 = hoàn hảo; 0.5 = đoán ngẫu nhiên. Tốt để so sánh mô hình.*

Đường cong Precision-Recall (PR)



- **PR curve:** đồ thị Precision theo Recall khi quét ngưỡng.
- **AP / PR-AUC:** diện tích dưới PR — tóm tắt chất lượng bằng một số.
- **Ưu tiên PR hơn ROC khi:** dữ liệu MẤT CÂN BẰNG mạnh (lớp dương hiếm), vì ROC có thể lạc quan quá mức.
- *Đường cơ sở = tỷ lệ lớp dương trong dữ liệu.*

Mất cân bằng lớp (Class Imbalance)



Lấy mẫu lại

Oversampling (SMOTE)
lớp hiếm hoặc
undersampling lớp đa

Trọng số lớp

`class_weight='balanced'`
— phạt nặng lỗi ở lớp
hiếm.

Dịch ngưỡng

Hạ ngưỡng để tăng
Recall cho lớp dương.

Đúng độ đo

Dùng F1, PR-AUC,
balanced accuracy —
KHÔNG dùng Accuracy.

- **Vấn đề:** khi một lớp áp đảo (vd 99% âm), Accuracy cao nhưng mô hình bỏ sót toàn bộ lớp hiếm.

05

Thực hành với Python

Hands-on with Python

Cài đặt NumPy: sigmoid, log loss, GD

```
import numpy as np

def sigmoid(z):
    return 1 / (1 + np.exp(-z))

def log_loss(y, h):
    # binary cross-entropy
    return -np.mean(y*np.log(h) + (1-y)*np.log(1-h))

# Gradient Descent
b = np.zeros(X.shape[1])
alpha, epochs = 0.1, 2000
for _ in range(epochs):
    h = sigmoid(X @ b)
    grad = X.T @ (h - y) / len(y)    # dL/db = (σ - y) · x
    b -= alpha * grad
```

scikit-learn: phân loại & báo cáo

```
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import confusion_matrix, classification_report

X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42)

clf = LogisticRegression(max_iter=1000, class_weight="balanced")
clf.fit(X_tr, y_tr)
y_pred = clf.predict(X_te)

print(confusion_matrix(y_te, y_pred))
print(classification_report(y_te, y_pred)) # P/R/F1 mỗi lớp
```

Điều chỉnh ngưỡng & vẽ ROC / PR

```
from sklearn.metrics import roc_curve, auc, precision_recall_curve
import matplotlib.pyplot as plt

proba = clf.predict_proba(X_te)[:, 1]      # xác suất lớp 1

# Điều chỉnh ngưỡng (mặc định 0.5)
threshold = 0.3
y_pred = (proba >= threshold).astype(int)

# Vẽ ROC
fpr, tpr, _ = roc_curve(y_te, proba)
plt.plot(fpr, tpr, label=f"AUC = {auc(fpr, tpr):.2f}")

# Vẽ PR
prec, rec, _ = precision_recall_curve(y_te, proba)
plt.plot(rec, prec)
```

Tóm tắt (Key Takeaways)

- **Logistic Regression** dùng sigmoid để ánh xạ đầu ra tuyến tính thành XÁC SUẤT trong (0, 1) rồi phân lớp theo ngưỡng.
- **Log Loss (Binary Cross-Entropy)** thay cho MSE vì tạo mặt tối ưu LỖI → đảm bảo hội tụ.
- **Gradient descent** với quy tắc chuỗi cho công thức gọn: $dL/db = (\sigma - y) \cdot x$.
- **Softmax & One-vs-Rest** tổng quát hóa cho đa lớp ($K > 2$).
- **Chọn độ đo khôn ngoan:** Accuracy cho dữ liệu cân bằng; Precision/Recall/F1 và PR-AUC cho dữ liệu mất cân bằng; AUC-ROC để so sánh độc lập ngưỡng.
- **Ngưỡng quyết định** là “nút vặn” điều chỉnh đánh đổi Precision $\rightleftharpoons$ Recall theo bài toán.